Screenshot2Spec

Complete Development Guide, Team Responsibilities & Ready-to-Use Antigravity Prompts

Tagline: See It. Specify It. Build It.

Project Category: Best Use of Gemma 4

AI Model: Gemma 4 via Google AI Studio API

Development Environment: Google Antigravity

Technology Stack: Next.js, TypeScript, Tailwind CSS, Google GenAI SDK, Zod, GitHub, Vercel

Team Size: 3 Members

Target Completion: 2–2.5 Hours

Maximum Available Time: 3 Hours, Including Testing and Deployment

---

1. Project Overview

1.1 What Is Screenshot2Spec?

Screenshot2Spec is an AI-powered developer productivity application that transforms user interface screenshots into implementation-ready frontend development blueprints.

Developers upload a screenshot of a website or application interface and optionally specify their preferred framework, styling system, and additional implementation requirements.

Google's Gemma 4 model analyzes the screenshot and identifies visible UI components, layout structures, design patterns, and potential user interactions.

The application then produces:

1. A structured UI analysis.
2. A proposed React component hierarchy.

3. Component-specific frontend library recommendations.
4. Required dependency installation commands.
5. Suggested TypeScript interfaces.
6. A frontend implementation checklist.
7. A comprehensive AI coding implementation prompt.
8. Downloadable developer documentation.

The generated implementation prompt can be copied into Google Antigravity, Codex, Cursor, Claude Code, or similar AI coding environments.

1.2 Problem Statement

Frontend developers frequently receive visual references without corresponding implementation specifications.

A screenshot illustrates how a UI looks, but does not explain:

- Which components should be created.
- How the components should be organized.
- Which libraries should implement charts, tables, forms, and navigation.
- Which dependencies need installation.
- What props and TypeScript interfaces are required.
- How the interface should behave across different screen sizes.
- How to describe the implementation accurately to an AI coding assistant.

Consequently, developers spend time manually analyzing screenshots, researching libraries, designing component architecture, and preparing prompts.

1.3 Proposed Solution

Screenshot2Spec converts visual references into structured frontend implementation instructions using Gemma 4.

Core Workflow:

Upload Screenshot → Gemma 4 Visual Analysis → Structured JSON Validation → Component Architecture → Library Recommendations → Implementation Prompt → Copy/Download

The application focuses on producing developer-ready specifications rather than automatically generating an entire application.

1.4 Unique Value Proposition

Screenshot2Spec does not stop at identifying UI elements. It explains how developers can implement them.

For example, if a screenshot contains an analytics dashboard, the application can recommend:

- shadcn/ui for cards, navigation, buttons, and dialogs.
- Recharts for charts and analytics visualizations.
- Lucide React for icons.
- Tailwind CSS for responsive layout and styling.
- TanStack Table when advanced table interactions are required.

The implementation prompt explains where these libraries should be used, which components are relevant, what dependencies are necessary, and how the finished UI should be validated.

Library recommendations are proposed implementation choices, not claims about which libraries the original website uses.

---

2. Final MVP Scope

2.1 Priority P0 — Mandatory Features

The application must support:

1. Screenshot upload using PNG, JPG, or WebP.
2. Screenshot preview.
3. Framework and styling preferences.
4. Optional user instructions.
5. Gemma 4 multimodal screenshot analysis.
6. Structured JSON parsing and Zod validation.
7. Component hierarchy generation.
8. Curated library recommendations.
9. Package installation instructions.
10. Complete AI implementation prompt generation.
11. Copy and download functionality.
12. Responsive results interface.
13. API error handling.
14. Production deployment on Vercel.

The project is not considered complete unless a real screenshot produces a usable, library-aware coding prompt.

2.2 Priority P1 — Important Features

Implement when the mandatory workflow is stable:

- TypeScript interface generation.
- Design specification export.
- Suggested folder structure.
- Prioritized development checklist.
- Separate dependency documentation.
- Basic automated generator tests.

2.3 Priority P2 — Future Scope

Do not implement during the initial 150-minute development target:

- User authentication.
- Database and project history.
- Figma integration.
- Automated frontend code execution.
- Automatic GitHub commits.
- Multiple screenshot comparisons.
- Before/after visual verification.
- IDE extensions.
- Complex drag-and-drop component editing.

---

3. Team Division and Responsibilities

Member 1 — Frontend and User Experience

Primary Responsibility: Build the application interface.

Tasks

1. Create the landing page.
2. Implement screenshot upload and preview.
3. Add framework and styling selectors.
4. Add optional instructions.
5. Connect the Analyze button to the backend.
6. Implement loading and error states.

7. Display screenshot analysis results.
8. Render the component hierarchy.
9. Display library recommendations.
10. Display the generated implementation prompt.
11. Implement copy and download buttons.
12. Ensure desktop and mobile responsiveness.

Required Deliverable

A fully functional frontend connected to `POST /api/analyze` that can display the validated analysis and generated developer blueprint.

Branch

`feature/frontend`

Member 2 — Gemma 4 API and Backend

Primary Responsibility: Implement secure multimodal AI integration.

Tasks

1. Configure the Google AI Studio API key.
2. Install the official Google GenAI SDK.
3. Implement `POST /api/analyze`.
4. Validate uploaded image format and file size.
5. Send screenshot and optional text to Gemma 4.
6. Write the image analysis instructions.
7. Parse the returned JSON.
8. Validate the response using Zod.
9. Handle model errors and malformed responses.
10. Return a consistent typed response to the frontend.
11. Test the API using real screenshots.

Required Deliverable

A working server-side Gemma API endpoint that returns validated screenshot analysis.

Branch

`feature/gemma-api`

Member 3 — Specification Generator, Integration and Deployment

Primary Responsibility: Convert validated analysis into actionable developer artifacts.

Tasks

1. Define the shared data contract.
2. Implement component hierarchy validation.
3. Create the curated frontend library catalog.
4. Match detected components with approved libraries.
5. Generate package installation instructions.
6. Generate the master AI implementation prompt.
7. Generate Markdown specifications.
8. Generate TypeScript interfaces when feasible.
9. Implement export functions.
10. Write generator unit tests.
11. Coordinate integration between frontend and backend.
12. Run production build verification.
13. Coordinate GitHub merging and Vercel deployment.

Required Deliverable

A deterministic specification generator that converts validated analysis into a developer-ready implementation blueprint.

Branch

`feature/spec-generator`

Shared Responsibilities

All three members must:

- Agree on the shared JSON schema before coding.
- Work in the same repository.
- Avoid changing files owned by other members without coordination.
- Integrate their branches early.
- Verify the end-to-end workflow.
- Test the deployed application.
- Prepare the hackathon demonstration.

---

4. Shared API Contract

This section must be agreed upon before parallel development begins.

4.1 API Endpoint

Endpoint: `POST /api/analyze`

Request Type: `multipart/form-data`

Request Fields

Field	Type	Required
screenshot	Image file	Yes
framework	String	Yes
styling	String	Yes
instructions	String	No

Supported formats: PNG, JPG and WebP.

Application-level file size limit: 5 MB.

4.2 Gemma Analysis Response

The API must return a structured object containing:

- `pageType`
- `summary`
- `layout`
- `components`
- `design`
- `interactions`
- `assumptions`

Each component should contain:

- `id`
- `name`

- `parentId`
- `uiType`
- `purpose`
- `observedEvidence`
- `confidence`
- `suggestedLibraryId`

The `confidence` field uses the categories `observed`, `inferred`, and `unknown`. These are evidence classifications, not numerical model-confidence scores.

The `suggestedLibraryId` must reference an approved catalog entry or be `null`.

4.3 Example JSON

None

```
{
  "pageType": "dashboard",
  "summary": "Analytics dashboard with sidebar and revenue
metrics",
  "layout": ["sidebar", "header", "main content"],
  "components": [
    {
      "id": "sidebar",
      "name": "AppSidebar",
      "parentId": null,
      "uiType": "navigation",
      "purpose": "Display navigation links",
      "observedEvidence": "Left vertical navigation region",
      "confidence": "observed",
      "suggestedLibraryId": "shadcn"
    },
    {
      "id": "revenue-chart",
      "name": "RevenueChart",
      "parentId": null,
      "uiType": "chart",
      "purpose": "Visualize revenue values",
      "observedEvidence": "Bar chart in the main content",
```

```
      "confidence": "observed",
      "suggestedLibraryId": "recharts"
    }
  ],
  "design": {
    "colors": ["#ffffff", "#172554"],
    "typography": ["Bold headings", "Muted secondary labels"],
    "spacing": ["Consistent card gaps"]
  },
  "interactions": [
    {
      "element": "Sidebar links",
      "behavior": "Navigate between pages",
      "status": "assumed"
    }
  ],
  "assumptions": [
    "Mobile navigation behavior is unknown",
    "The chart data source cannot be determined"
  ]
}
```

This is an illustrative contract. The team should implement the corresponding Zod schema and TypeScript types in a shared module.

4.4 Validation Rules

The application must verify:

1. All required fields exist.
2. Component IDs are unique.
3. Each `parentId` is either `null` or references an existing component.
4. The component hierarchy contains no cycles.
5. The library ID exists in the approved catalog or is `null`.
6. Component names can be safely normalized for generated TypeScript.
7. Missing information is not fabricated.
8. Unknown interactions remain explicitly identified.
9. Invalid responses return controlled errors.

Important: Do not maintain separate conflicting response structures in different team members' code.

---

5. Intelligent Library Recommendation System

5.1 Purpose

The library recommendation engine translates screenshot-derived component requirements into specific implementation technologies.

The system must recommend libraries based on actual UI needs rather than adding packages indiscriminately.

5.2 Approved Library Catalog

UI Requirement	Library	Package or Approach
Basic layouts	Tailwind CSS	Existing styling configuration
Cards and buttons	shadcn/ui	Card, Button
Sidebar navigation	shadcn/ui	Sidebar
Dialogs and dropdowns	shadcn/ui	Dialog, Select
Icons	Lucide React	lucide-react
Charts	Recharts	recharts
Advanced tables	TanStack Table	@tanstack/react-table
Validated forms	React Hook Form + Zod	react-hook-form, zod, @hookform/resolvers
Flowcharts	React Flow	@xyflow/react
Animations	Motion	motion
Simple tables	Native HTML	table, thead, tbody

Responsive layouts	Tailwind CSS	Grid, Flexbox

5.3 Library Selection Rules

1. Prefer libraries already installed in the target project.
2. Prefer native HTML and Tailwind for simple elements.
3. Use shadcn/ui only for components that benefit from reusable primitives.
4. Recommend Recharts only when data visualizations require it.
5. Recommend TanStack Table only for advanced table behavior.
6. Avoid complex dependencies for simple visual elements.
7. Never invent package names.
8. Recommend libraries from the approved catalog only.
9. Identify libraries as required or optional.
10. Explain the selected components and their purpose.

5.4 Example Generated Library Recommendations

For a dashboard containing a sidebar, four statistic cards, a revenue graph, and a simple table:

Sidebar: shadcn/ui Sidebar.

Statistic Cards: shadcn/ui Card.

Revenue Graph: Recharts BarChart.

Orders Table: Native semantic HTML unless sorting or filtering is explicitly required.

Icons: Lucide React.

Layout: Tailwind CSS Grid and Flexbox.

5.5 Example Installation Instructions

For a new compatible Next.js project:

```
None
npx shadcn@latest init
npx shadcn@latest add sidebar card button
npm install lucide-react recharts
```

Generated prompts must instruct coding agents to inspect `package.json` and existing components before installing packages.

Commands must be taken from the verified catalog, not invented by Gemma.

---

6. AI Implementation Prompt Generator

6.1 Main Purpose

The most important output of Screenshot2Spec is a complete implementation prompt suitable for an AI coding agent.

It must be specific enough to guide an implementation without requiring the developer to manually research every UI component.

6.2 Mandatory Prompt Sections

Every generated prompt must include:

1. Role and implementation objective.
2. Reference screenshot instructions.
3. Target framework and styling preferences.
4. Identified layout and component architecture.
5. Recommended libraries for each component.
6. Dependency installation instructions.
7. TypeScript interface recommendations.
8. Suggested file and folder organization.
9. Design and styling instructions.
10. Responsive behavior.
11. Interaction assumptions.
12. Accessibility requirements.
13. Implementation sequence.
14. Validation and testing steps.
15. Expected deliverables.

6.3 Sample Generated Master Prompt

Role

You are a senior React and TypeScript frontend engineer experienced in implementing responsive, maintainable interfaces from visual references.

Objective

Implement the uploaded interface screenshot using the selected technology stack.

Use the original screenshot as the primary visual reference.

Recreate the visible design as closely as practical while following maintainable component architecture.

Technology Stack

Use React, TypeScript, Next.js App Router, and Tailwind CSS.

Use the following libraries where justified by the screenshot:

- shadcn/ui for reusable UI components.
- Lucide React for icons.
- Recharts for analytics charts.
- Native HTML for basic tables.

Component Architecture

Create the following proposed components:

- DashboardPage
- AppSidebar
- DashboardHeader
- MetricsGrid
- MetricCard
- RevenueChart
- RecentOrdersTable

Use appropriate component props and reusable patterns.

Installation

Inspect existing dependencies first.

Install missing required packages only.

For a new compatible project, use the verified installation commands included in the generated dependency plan.

Data Models

Create TypeScript interfaces for visible data-driven components.

Use typed mock data unless a backend contract is explicitly provided.

Do not invent endpoints or external integrations.

Visual Requirements

Match the reference screenshot's:

- Layout structure.
- Color palette.
- Typography hierarchy.
- Spacing and alignment.
- Component dimensions where visually inferable.
- Borders and surface treatments.
- Icon placement.
- Chart appearance.

Responsive Behavior

Implement reasonable desktop, tablet, and mobile layouts.

Label any inferred responsive behavior as an implementation assumption.

Implementation Steps

1. Inspect the repository.
2. Review existing dependencies.
3. Set up only missing required libraries.
4. Define data models.
5. Build the primary layout.
6. Implement reusable components.
7. Implement data-driven elements.
8. Apply styling.
9. Add required interactions.
10. Test responsive behavior.
11. Run lint, type-checking, and production build commands.
12. Fix verified issues.

Validation

Check visual similarity, TypeScript correctness, responsiveness, accessibility, and functional interactions.

Do not claim tests have passed unless executed.

Deliverables

- Working frontend implementation.
- Reusable components.
- TypeScript interfaces.
- Required dependency changes.
- Summary of modified files.
- Test results.
- Remaining assumptions.

Final Instruction

Do not modify unrelated project files or introduce unnecessary dependencies.

Prioritize visual fidelity, maintainability, and the library recommendations in the specification.

6.4 Prompt Generation Architecture

Use one Gemma analysis request.

After validating the analysis, generate the implementation prompt using ordinary TypeScript functions and predefined templates.

Do not make additional AI requests for the MVP unless essential.

This reduces latency, cost, and inconsistent outputs.

---

7. Application Architecture

7.1 Processing Flow

1. User uploads a screenshot.
2. User selects development preferences.
3. Frontend sends multipart FormData to `/api/analyze`.
4. Backend validates the uploaded image.

5. Backend submits image and instructions to Gemma 4.
6. Gemma returns structured screenshot analysis.
7. Backend parses and validates the JSON with Zod.
8. Specification generator constructs component relationships.
9. Library engine matches approved dependencies.
10. Prompt generator assembles the implementation instructions.
11. Frontend displays the result.
12. User copies or downloads the generated developer artifacts.

7.2 Technology Stack

Layer	Technology
Frontend	Next.js
Language	TypeScript
Styling	Tailwind CSS
AI	Gemma 4
API Provider	Google AI Studio
SDK	@google/genai
Validation	Zod
Components	React
Icons	Lucide React
Deployment	Vercel
Version Control	GitHub

7.3 Recommended Project Structure

```
None
screenshot2spec/
├── src/
│   ├── app/
│   │   ├── page.tsx
│   │   ├── layout.tsx
```

```
|   |   ├── globals.css
|   |   └── api/
|   |       └── analyze/
|   |           └── route.ts
|   ├── components/
|   |   ├── ScreenshotUpload.tsx
|   |   ├── ResultsDashboard.tsx
|   |   ├── ComponentTree.tsx
|   |   ├── LibraryRecommendations.tsx
|   |   ├── PromptPreview.tsx
|   |   └── ExportActions.tsx
|   ├── lib/
|   |   ├── gemma.ts
|   |   ├── prompts.ts
|   |   ├── schemas.ts
|   |   ├── libraryCatalog.ts
|   |   ├── specificationGenerator.ts
|   |   └── exporters.ts
|   └── types/
|       └── specification.ts
├── tests/
├── public/
├── .env.example
├── .gitignore
├── README.md
└── package.json
```

---

8. Parallel Development Using Google Antigravity

8.1 Development Strategy

Screenshot2Spec will be developed by three team members using Google Antigravity, GitHub, and feature branches.

The project must be completed, tested, and deployed within three hours, with a target implementation time of 150 minutes.

Development will follow three phases:

Phase 1 — Shared Foundation (0–10 minutes)

All members agree on the API contract, TypeScript types, file ownership, and integration responsibilities.

Phase 2 — Parallel Development (10–70 minutes)

Each member works independently on their assigned branch:

- Member 1: Frontend and user interface.
- Member 2: Gemma 4 API integration and backend.
- Member 3: Specification generator, library recommendation engine, and implementation prompt generation.

Phase 3 — Integration, Testing, and Deployment (70–150 minutes)

Merge completed features, connect the end-to-end workflow, test the application, and deploy to Vercel.

Important Parallel Development Rules

1. All members work from the same initialized repository.
2. All members use the same shared TypeScript and Zod contracts.
3. Each member works on an independent feature branch.
4. No member changes another member's owned files without coordination.
5. Antigravity must inspect existing code before changing files.
6. Each member runs relevant checks before pushing changes.
7. Do not introduce unrelated features.
8. Do not expose the Google AI Studio API key.
9. The specification generator must not depend on additional Gemma API calls.
10. The integration owner merges work only after reviewing interfaces and conflicts.

8.2 Shared Foundation Prompt — Run First

Owner: Member 3

Duration: First 10 minutes

Branch: `main` during initial setup

Objective: Establish the common architecture and contracts before parallel development.

Ready-to-Paste Antigravity Prompt

You are the lead software architect for Screenshot2Spec, a three-member Gemma 4 hackathon project.

PROJECT OVERVIEW:

Screenshot2Spec converts UI screenshots into developer-ready frontend blueprints using Gemma 4 through Google AI Studio.

The application analyzes visible UI elements, generates component hierarchies, recommends appropriate frontend libraries, produces TypeScript interfaces, and creates implementation prompts for AI coding agents.

TECH STACK:

- Next.js App Router
- TypeScript
- Tailwind CSS
- Google GenAI SDK
- Zod
- GitHub
- Vercel

YOUR TASK:

Inspect the current repository.

If the project already exists, preserve its structure and working code.

If the project is empty, initialize a minimal Next.js application.

Create the shared development foundation only.

REQUIRED IMPLEMENTATION:

1. Create `src/types/analysis.ts`.

Define the shared analysis types, including:

- pageType

- summary
- layout
- components
- design
- interactions
- assumptions

Each component must include:

- id
- name
- parentId
- uiType
- purpose
- observedEvidence
- confidence
- suggestedLibraryId

Use the confidence classifications:

- observed
- inferred
- unknown

2. Create `src/types/specification.ts`.

Define the developer blueprint structure:

- componentTree
- designSummary
- libraryRecommendations
- installationCommands
- suggestedFolderStructure
- typescriptOutline
- implementationChecklist
- codingAssistantPrompt
- assumptions

3. Create `src/lib/schemas.ts`.

Define Zod validation for the shared analysis structure.

The schema must verify required field types.

Add separate integrity checks for duplicate component IDs, unknown parent references, and circular component relationships.

4. Create `src/lib/mockAnalysis.ts`.

Provide realistic mock data representing an analytics dashboard with sidebar navigation, metric cards, a revenue chart, and a recent orders table.

5. Create `.env.example`.

Include:

GEMINI_API_KEY=your_api_key_here

GEMMA_MODEL=gemma-4-26b-a4b-it

6. Ensure `.env.local` and other private environment files are ignored by Git.
7. Ensure TypeScript, Tailwind CSS, and required dependencies are configured.
8. Create a short `docs/TEAM_CONTRACT.md` explaining the shared API response and which files each teammate owns.

API CONTRACT:

POST /api/analyze

Request: multipart/form-data

Fields:

- screenshot: required image
- framework: string
- styling: string
- instructions: optional string

Response on success:

{
"analysis": ValidatedAnalysis
}

Do not generate the implementation blueprint inside the API route. The frontend or a separate orchestration function will pass the validated analysis to the specification generator.

FILE OWNERSHIP:

Member 1 owns:

- src/app/page.tsx
- src/app/globals.css
- src/components/**

Member 2 owns:

- src/app/api/analyze/route.ts
- src/lib/gemma.ts
- src/lib/analysisPrompt.ts

Member 3 owns:

- src/lib/libraryCatalog.ts
- src/lib/specificationGenerator.ts
- src/lib/promptGenerator.ts
- src/lib/exporters.ts
- tests/**

Shared types and schemas are locked after this initial setup. Any later changes require team agreement.

RESTRICTIONS:

- Do not build the complete application.
- Do not implement frontend features.
- Do not implement Gemma API calls.
- Do not implement the specification generator.
- Do not add authentication or database functionality.
- Do not change unrelated repository files.

VALIDATION:

Run the available TypeScript and build checks.

Fix any setup-related errors.

DELIVERABLES:

- Working project foundation
- Shared types and schemas
- Mock analysis data
- Team contract document
- Initial commit on main

After completing the foundation, report:

1. Files created.
2. Dependencies installed.
3. Commands executed.
4. Actual validation results.
5. Any unresolved blockers.

Do not wait for additional approval between setup steps unless an essential requirement is missing.

8.3 Member 1 Prompt — Frontend and UI Development

Owner: Member 1

Branch: `feature/frontend`

Start: Immediately after the shared foundation is committed.

Objective: Build a polished, responsive Screenshot2Spec interface using the shared mock analysis and specification contracts.

Ready-to-Paste Antigravity Prompt

You are the senior frontend engineer responsible for Screenshot2Spec.

You are working in parallel with two other developers.

YOUR RESPONSIBILITY:

Implement only the frontend user experience.

Do not implement or modify Gemma API logic, Zod schemas, shared types, or the specification generator.

TECH STACK:

- Next.js App Router
- React
- TypeScript
- Tailwind CSS
- Lucide React

FIRST:

Inspect the existing repository, shared types, and mock analysis data.

Review `docs/TEAM_CONTRACT.md`.

Create or switch to the branch:

feature/frontend

Preserve all existing shared contracts.

BUILD THE FOLLOWING:

1. Landing Page

Create a modern developer-focused landing page.

Include:

- Screenshot2Spec branding
- Tagline: "See It. Specify It. Build It."
- Short product description
- Screenshot upload interface
- Framework selection
- Styling selection
- Optional development instructions
- Analyze button

2. Screenshot Upload

Support:

- PNG
- JPEG
- WebP

Allow a maximum of 5 MB.

Show image preview and filename.

Provide a remove/replace action.

Reject unsupported formats with descriptive messages.

3. Framework Selection

For the MVP, the supported implementation target is:

React + TypeScript using Next.js App Router.

Do not offer unsupported frameworks as working options.

4. Styling Selection

Default to Tailwind CSS.

If additional styling options are displayed, clearly mark them as unavailable until implemented.

5. Analysis Request

When the user clicks Analyze:

- Construct FormData.
- Append screenshot, framework, styling, and instructions.
- POST to `/api/analyze`.
- Read the returned analysis object.
- Pass the validated result to the shared specification generator.

Do not manually set the multipart Content-Type header.

6. Results Dashboard

Create sections or tabs for:

- Overview
- Component Hierarchy
- Library Recommendations
- Installation Commands
- TypeScript Interfaces
- Implementation Checklist
- AI Coding Prompt

7. Component Hierarchy

Display parent-child relationships clearly.

Use a recursive renderer or equivalent.

Handle empty and malformed display data safely.

8. Library Recommendations

Display:

- Recommended library
- Relevant component
- Purpose
- Reason for selection
- Required or optional status

9. AI Coding Prompt Viewer

Provide a dedicated section displaying the generated implementation prompt.

Add:

- Copy Prompt
- Download Prompt
- Clear Result

10. Export Actions

Connect available export functions to download the developer artifacts.

11. Application States

Implement:

- Idle
- Image selected
- Analyzing
- Success
- Validation error
- API failure

Disable duplicate submission during analysis.

12. Responsive Design

Ensure the application works on:

- Desktop
- Tablet
- Mobile

Use a polished developer-tool visual style with clear typography and compact information panels.

Prioritize usability over excessive animations.

INTEGRATION RULES:

Use shared types from `src/types`.

Use the shared mock analysis until the real API is ready.

Import generator functions from their agreed module paths.

Do not copy the specification generator into frontend components.

Do not modify files owned by Member 2 or Member 3.

If a required generator function is not implemented yet, use a temporary typed mock adapter in a frontend-owned file and remove it during integration.

SECURITY:

Never put GEMINI_API_KEY in client code.

Do not store API credentials in browser storage.

VALIDATION:

Run the available ESLint, TypeScript, and production build checks.

Fix frontend-related errors.

Test screenshot selection, preview, loading states, result rendering, copy actions, and downloads.

DELIVERABLE:

A responsive Screenshot2Spec frontend ready to connect to the real API and specification generator.

When complete:

- Commit changes.
- Push feature/frontend.
- Report files changed.
- Report tests and checks actually run.
- Report any integration requirements.

Do not modify backend or generator-owned files.

8.4 Member 2 Prompt — Gemma 4 API and Backend

Owner: Member 2

Branch: `feature/gemma-api`

Start: Immediately after the shared foundation is committed.

Objective: Implement a secure, reliable screenshot analysis endpoint powered by Gemma 4 through Google AI Studio.

Ready-to-Paste Antigravity Prompt

You are the backend and AI integration engineer for Screenshot2Spec.

You are working in parallel with two other developers.

YOUR RESPONSIBILITY:

Implement the Gemma 4 multimodal analysis pipeline and API endpoint.

Do not modify frontend components or specification generator modules.

FIRST:

Inspect the existing repository.

Read:

- src/types/analysis.ts
- src/lib/schemas.ts
- docs/TEAM_CONTRACT.md

Create or switch to:

feature/gemma-api

TECHNOLOGY:

- Next.js Route Handlers
- TypeScript
- @google/genai
- Gemma 4 through Google AI Studio
- Zod

REQUIRED IMPLEMENTATION:

1. Create `src/lib/gemma.ts`.

Initialize the Google GenAI client using:

GEMINI_API_KEY

Read the key only from server-side environment variables.

Use the model ID configured through GEMMA_MODEL.

Verify the supported model identifier and image-request format against official Google documentation or the installed SDK before implementing the call.

2. Create `src/lib/analysisPrompt.ts`.

Write a screenshot analysis prompt instructing Gemma 4 to identify:

- Page type
- Page summary
- Layout regions
- Visible components
- Component hierarchy
- Design observations
- Potential interactions
- Assumptions

Each component must have the fields defined in the shared contract.

3. Implement `POST /api/analyze`.

Location:

src/app/api/analyze/route.ts

Accept multipart FormData.

Validate:

- Required screenshot
- PNG/JPEG/WebP MIME type
- Maximum size 5 MB
- Supported framework and styling values
- Optional instructions within a reasonable length limit

4. Image Processing

Convert validated image bytes into the supported image-part format for the Google GenAI SDK.

Send both the screenshot and text instructions in a single model request.

5. Model Output

Require the model to respond with a JSON object matching the shared analysis contract.

Rules for the model:

- Do not invent invisible UI elements.
- Do not invent backend integrations.
- Do not infer exact implementation libraries from appearance.
- Distinguish visible details from assumptions.
- Use approved library IDs or null.
- Use unique component IDs.
- Provide grounded observations.
- Mark unknown behavior clearly.
- Do not return executable source code.

6. Response Validation

Parse the model response safely.

Remove only recognized outer Markdown code fences if necessary.

Validate the result using the shared Zod schema.

Run component hierarchy integrity checks.

If JSON parsing or validation fails, return a controlled response.

Do not fabricate missing analysis fields.

7. API Response

Return:

```
{
"analysis": validatedData
}
```

Do not add a different response structure.

8. Error Handling

Handle:

- Missing image
- Invalid MIME type
- Oversized image
- Missing API key
- Unavailable model
- API quota errors
- Network failures
- Empty model response
- Invalid JSON

- Schema mismatch

Use appropriate HTTP status codes and user-friendly error messages.

Never expose internal secrets in errors.

9. Logging

Log limited server-side diagnostic information without printing API keys or uploaded image contents.

10. Testing

Test using:

- One simple landing page screenshot
- One dashboard screenshot
- One form-based screenshot
- Invalid file type
- Oversized file
- Missing API key

Run TypeScript and build checks.

Verify the API response matches the shared contract.

FILE OWNERSHIP:

You may modify:

- src/app/api/analyze/route.ts
- src/lib/gemma.ts
- src/lib/analysisPrompt.ts

Do not modify:

- src/app/page.tsx
- src/components/**
- src/lib/specificationGenerator.ts
- src/lib/libraryCatalog.ts
- src/lib/promptGenerator.ts
- src/lib/exporters.ts

Do not change shared types without communicating the proposed change to the team.

DELIVERABLE:

A tested Gemma-powered screenshot analysis endpoint returning validated structured JSON.

When complete:

- Commit changes.
- Push feature/gemma-api.
- Report files changed.
- Report successful and failed test results.
- Document required environment variables.
- Explain any remaining API limitations.

Do not claim the endpoint works unless a real model call succeeds.

8.5 Member 3 Prompt — Specification, Library Mapping and Prompt Generation

Owner: Member 3

Branch: `feature/spec-generator`

Start: Immediately after the shared foundation is committed.

Objective: Build the deterministic blueprint generator responsible for the project's main differentiator.

Ready-to-Paste Antigravity Prompt

You are the specification engineering and developer tooling lead for Screenshot2Spec.

You are working in parallel with two other developers.

YOUR RESPONSIBILITY:

Convert validated Gemma screenshot analysis into implementation-ready frontend blueprints.

Do not implement frontend pages or the Gemma API endpoint.

FIRST:

Inspect the repository.

Read:

- src/types/analysis.ts
- src/types/specification.ts
- src/lib/schemas.ts
- src/lib/mockAnalysis.ts
- docs/TEAM_CONTRACT.md

Create or switch to:

feature/spec-generator

MAIN OBJECTIVE:

Given a validated analysis object, generate:

1. Component hierarchy
2. Design summary
3. Component-specific library recommendations
4. Dependency installation commands
5. Suggested folder structure
6. TypeScript interface outline
7. Implementation checklist
8. AI coding implementation prompt
9. Exportable Markdown and JSON artifacts

REQUIRED IMPLEMENTATION:

A. Library Catalog

Create:

src/lib/libraryCatalog.ts

Define a curated catalog containing:

- Tailwind CSS
- Native HTML
- shadcn/ui
- Lucide React
- Recharts
- TanStack Table
- React Hook Form
- Zod
- React Flow
- Motion

Each entry should include:

- ID
- Library name
- Package name
- Supported UI categories
- Relevant components
- Recommended use cases
- Installation instructions
- Official documentation URL
- Optional/required recommendation criteria

Never generate installation commands from unverified model output.

B. Component-to-Library Matching

Match UI component types against the catalog.

Examples:

- Chart → Recharts
- Icons → Lucide React
- Basic grid → Tailwind CSS
- Simple table → Native HTML
- Interactive data table → TanStack Table
- Dialog → shadcn/ui
- Node diagram → React Flow

Use conservative matching.

Do not introduce an advanced library unless the screenshot and requirements justify it.

C. Specification Generator

Create:

src/lib/specificationGenerator.ts

Input:

Validated Analysis object.

Output:

Specification object matching the shared type.

Generate:

- Readable component tree

- Layout and design summary
- Library recommendation list
- Dependency plan
- Folder structure
- TypeScript outline
- Implementation checklist
- Assumptions

Use ordinary TypeScript logic.

Do not call Gemma or any other AI model.

D. Master Implementation Prompt Generator

Create:

src/lib/promptGenerator.ts

Generate a complete coding-agent-ready prompt containing:

1. Role and objective
2. Reference screenshot usage
3. Framework and styling instructions
4. Component breakdown
5. Approved library selection
6. Component-specific library usage
7. Verified installation commands
8. Suggested file structure
9. Proposed TypeScript interfaces
10. Design and layout instructions
11. Responsive behavior guidance
12. Interaction assumptions
13. Accessibility requirements
14. Implementation sequence
15. Validation requirements
16. Expected deliverables

The generated prompt must explain not only which libraries to install but which components should use them and why.

The prompt should direct the target coding agent to inspect its existing repository before making changes.

It must not claim the screenshot reveals hidden backend implementation details.

E. TypeScript Generation

Generate simple, syntactically valid TypeScript interfaces for suggested component data.

Use a restricted type set.

Sanitize generated names.

Do not execute the generated code.

F. Export Functions

Create:

src/lib/exporters.ts

Implement functions for:

- Implementation prompt Markdown
- Frontend specification Markdown
- Dependency plan Markdown
- Component tree JSON
- TypeScript interfaces

Keep these functions reusable by the frontend.

G. Automated Tests

Create focused tests for:

- Valid component hierarchy
- Duplicate IDs
- Invalid parent references
- Circular relationships
- Library catalog matching
- Unsupported library IDs
- Dependency deduplication
- Prompt section completeness
- Export output validity

INTEGRATION:

Expose typed functions that Member 1 can import directly.

Do not change shared function contracts without coordination.

Do not modify backend or frontend-owned files.

RESTRICTIONS:

- No additional model calls.
- No database.
- No authentication.
- No unnecessary dependencies.
- No executable generated code.
- No invented package names.
- No arbitrary remote code or script execution.

VALIDATION:

Run relevant TypeScript checks and unit tests.

Confirm the generator works using the shared mock analysis data.

DELIVERABLE:

A tested deterministic specification generator producing complete library-aware implementation prompts and downloadable artifacts.

When complete:

- Commit changes.
- Push feature/spec-generator.
- Report exported functions.
- Report changed files.
- Report tests actually run.
- Identify any integration requirements.

8.6 Parallel Integration Prompt — Run After All Three Branches Are Ready

Owner: Member 3 coordinates; all three members participate.

Branch: `main` or a dedicated `integration` branch.

Ready-to-Paste Antigravity Prompt

You are the integration engineer for Screenshot2Spec.

Three parallel development branches have been created:

- feature/frontend
- feature/gemma-api
- feature/spec-generator

Your task is to integrate these features into a stable application.

FIRST:

Inspect the current repository, branch history, and shared contracts.

Do not blindly overwrite files during merges.

Review any conflicts and preserve the intended functionality of each branch.

REQUIRED END-TO-END FLOW:

1. User uploads a screenshot.
2. Frontend sends FormData to POST /api/analyze.
3. Backend validates the upload.
4. Gemma 4 analyzes the image and instructions.
5. Backend validates the structured response.
6. Specification generator creates the component hierarchy and library plan.
7. Prompt generator creates a complete implementation prompt.
8. Frontend displays the outputs.
9. User copies or downloads the artifacts.

VERIFY:

- Request field names match.
- Response shape matches the shared contract.
- All imports resolve.
- No duplicated generator logic exists.
- No model calls occur in the deterministic generator.
- Library recommendations use the approved catalog.
- Installation commands are valid.
- Copy and download controls work.
- Loading and error states work.
- Environment variables remain server-side.

RUN:

- TypeScript checks
- ESLint
- Production build
- Available unit tests
- End-to-end manual testing with a real screenshot

FIX:

Only critical bugs, broken integration points, and production-blocking errors.

DO NOT:

- Rewrite the entire application.
- Add a database.
- Add authentication.
- Change the visual design unnecessarily.
- Introduce unrelated dependencies.
- Add optional features before production works.

DELIVERABLE:

A stable Screenshot2Spec application ready for deployment.

Report:

1. Merge and integration status.
2. Files changed.
3. Tests executed and results.
4. Remaining limitations.
5. Deployment readiness.

8.7 Deployment Prompt — Vercel Production Release

Owner: Member 3

Timing: After integration and critical tests.

Ready-to-Paste Antigravity Prompt

Act as the release engineer for Screenshot2Spec.

Prepare the integrated Next.js application for production deployment on Vercel.

TASKS:

1. Inspect package.json and deployment configuration.
2. Run the production build.

3. Verify server-side environment variable usage.
4. Ensure GEMINI_API_KEY is not exposed to client code.
5. Confirm `.env.local` is excluded from Git.
6. Verify `GEMMA_MODEL` configuration.
7. Review API request size and serverless compatibility.
8. Ensure no local filesystem persistence is required.
9. Confirm the application uses supported Next.js server-side APIs.
10. Ensure README and `.env.example` are accurate.
11. Check the public repository for accidentally committed credentials.
12. Confirm the complete application is ready for Vercel.

If Vercel CLI is installed and authenticated, use it only with the authorized project and environment.

Otherwise, provide the exact Vercel dashboard steps without claiming deployment succeeded.

After deployment, test:

- Homepage
- Screenshot upload
- Real Gemma analysis
- Component hierarchy
- Library recommendations
- Implementation prompt
- Copy/export functionality
- Mobile layout
- API error handling

Do not claim deployment is complete until the production URL has been opened and the main workflow has been tested.

Provide the final deployment status, URL if available, and any unresolved issues.

8.8 Final QA and Hackathon Audit Prompt

Owner: All three members

Ready-to-Paste Antigravity Prompt

Act as a senior QA engineer, frontend architect, and hackathon evaluator.

Review Screenshot2Spec against its defined MVP and Best Use of Gemma 4 objectives.

PROJECT GOAL:

Transform a UI screenshot into a validated, library-aware frontend specification and a complete implementation prompt.

TEST THE FULL WORKFLOW:

Screenshot Upload → Gemma 4 Analysis → JSON Validation → Component Architecture → Library Selection → Dependency Plan → Implementation Prompt → Export.

AUDIT CATEGORIES:

1. Functional correctness
2. Gemma 4 integration
3. Multimodal image understanding
4. Shared TypeScript contracts
5. Zod validation
6. Component hierarchy integrity
7. Library catalog accuracy
8. Installation command correctness
9. Prompt completeness
10. TypeScript generation
11. Export functionality
12. Frontend responsiveness
13. API and secret security
14. Error handling
15. Vercel production readiness
16. README and documentation

TEST CASES:

- Simple landing page
- Analytics dashboard
- Form-based interface
- Invalid image
- Oversized image
- Missing API key
- Invalid JSON response
- API timeout or quota error
- Empty components
- Nested component hierarchy
- Unsupported library recommendation

VALIDATE GENERATED PROMPTS:

Each prompt must contain:

- Target technology stack
- Component hierarchy
- Component-specific library recommendations
- Installation instructions
- TypeScript guidance
- Layout requirements
- Responsive behavior
- Accessibility requirements
- Implementation steps
- Testing instructions
- Expected deliverables
- Uncertainty warnings

IMPORTANT:

Only report test results that were actually observed.

Do not invent successful API calls, builds, deployments, or test results.

Do not add new features during the final audit.

Fix only issues that block the MVP.

FINAL REPORT:

1. Critical bugs fixed.
2. Remaining issues.
3. Tests passed.
4. Tests failed or not executed.
5. Production deployment status.
6. Readiness for the hackathon demonstration.

8.9 Team Handoff Checklist

Before integration, each member must confirm their responsibilities are complete.

Member 1

- Landing page completed.
- Screenshot upload working.
- API request implemented.

- Results dashboard implemented.
- Component tree displayed.
- Library recommendations displayed.
- Prompt viewer implemented.
- Copy/download controls implemented.
- Responsive layout checked.
- Frontend branch pushed.

Member 2

- Google AI Studio configured.
- Gemma 4 request succeeds.
- POST /api/analyze implemented.
- File validation implemented.
- JSON parsing implemented.
- Zod validation implemented.
- Error handling tested.
- API key security verified.
- Real screenshot tested.
- Backend branch pushed.

Member 3

- Shared contract established.
- Library catalog implemented.
- Component hierarchy generator completed.
- Installation commands generated.
- TypeScript outlines generated.
- Master implementation prompt completed.
- Export functions completed.
- Unit tests executed.
- Generator branch pushed.
- Integration and deployment prepared.

Final Team Check

- All three branches merged.
- No unresolved merge conflicts.
- Production build succeeds.
- Real screenshot produces correct results.
- Generated prompt is useful and complete.
- Vercel deployment works.

- README and license included.
- Final demo rehearsed.

Final Rule: No optional features until the entire P0 workflow works on the deployed application.

---

9. Development Timeline — 150 Minutes

Time	Activity	Responsible
00–10 min	Repository setup and shared contract	All members
10–40 min	Parallel frontend, backend, and generator implementation	Separate owners
40–70 min	Connect Gemma and generate a complete specification	All members
70–100 min	Complete UI, library recommendations, and implementation prompt	All members
100–120 min	Functional testing and critical fixes	All members
120–140 min	Merge, production build, and Vercel deployment	Member 3 leads
140–150 min	Final smoke test, README, and demo preparation	All members

Additional contingency: 30 minutes within the full three-hour window.

Mandatory Checkpoints

Minute 10: Shared schema finalized.

Minute 40: Frontend, backend, and specification generator each have a working initial implementation.

Minute 70: Real screenshot can travel through the complete application.

Minute 100: Library-aware implementation prompt and downloads work.

Minute 120: Critical tests pass and the application is ready for deployment.

Minute 140: Live Vercel URL is accessible.

Minute 150: Demonstration and submission materials are ready.

If a checkpoint is missed, stop adding optional features.

---

10. Testing and Quality Assurance

10.1 Functional Tests

- Upload valid PNG, JPG, and WebP screenshots.
- Reject invalid file formats.
- Reject files exceeding the configured size limit.
- Analyze a landing page screenshot.
- Analyze an analytics dashboard screenshot.
- Analyze a form-based UI screenshot.
- Handle invalid model JSON.
- Handle empty and malformed model responses.
- Handle API quota or connectivity failures.
- Verify component hierarchy generation.
- Verify library recommendations use approved catalog entries.
- Verify installation command generation.
- Verify coding prompt completeness.
- Verify copy and export actions.
- Verify mobile responsiveness.

10.2 Library Recommendation Tests

Check that:

- A basic card does not require an unnecessary dependency.
- A chart receives an appropriate chart-library recommendation.
- A simple table can use native HTML.
- An advanced data table can recommend TanStack Table.
- Package names come from the approved catalog.
- Installation instructions do not duplicate existing dependencies.
- Unsupported recommendations are rejected.

10.3 AI Prompt Quality Tests

Every generated implementation prompt must include:

- Screenshot reference instructions.
- Selected technology stack.
- Proposed component hierarchy.
- Specific library recommendations.
- Dependency installation commands.
- TypeScript implementation guidance.
- Responsive layout requirements.
- Accessibility expectations.
- Implementation sequence.
- Validation instructions.
- Required deliverables.
- Clearly labeled assumptions.

10.4 Security and Reliability

- Keep GEMINI_API_KEY server-side.
- Never log sensitive API credentials.
- Validate file type and size before model invocation.
- Limit user-provided instruction length.
- Treat uploaded content and model output as untrusted.
- Avoid executing generated code.
- Keep generated files downloadable rather than automatically applying them.
- Do not claim unexecuted tests passed.

10.5 Production Acceptance Criteria

Screenshot2Spec is ready when:

1. The production URL loads successfully.
2. A real screenshot can be uploaded.
3. Gemma 4 returns a usable analysis.
4. The component tree renders correctly.
5. Library recommendations are valid and relevant.
6. The implementation prompt contains actionable instructions.
7. The prompt can be copied and downloaded.
8. Errors are handled clearly.
9. The project builds successfully.
10. The GitHub repository contains setup instructions and a license.

---

11. Deployment Plan — GitHub + Vercel

Step 1 — Local Setup

Create the project if it does not already exist:

```
None
npx create-next-app@latest screenshot2spec --ts --tailwind --eslint
--app --src-dir
cd screenshot2spec
npm install @google/genai zod lucide-react
```

Step 2 — Configure Environment Variables

Create `.env.local`:

```
None
GEMINI_API_KEY=your_google_ai_studio_api_key
GEMMA_MODEL=gemma-4-26b-a4b-it
```

Confirm the model is available in the project's Google AI Studio account.

Never commit `.env.local`.

Step 3 — Run Locally

```
None
npm run dev
```

Test screenshot upload and the real Gemma API response.

Step 4 — Run Build Verification

```
None
npm run lint
npm run build
```

If the installed framework version does not support `npm run lint` by default, use the project's configured ESLint or equivalent validation command.

Resolve critical errors before deployment.

Step 5 — Push to GitHub

```
None
git init
git add .
git commit -m "Build Screenshot2Spec Gemma 4 MVP"
git branch -M main
```

Create a public repository, add its remote URL, and push the branch.

Ensure README, LICENSE, and `.env.example` are included.

Step 6 — Deploy on Vercel

1. Open https://vercel.com/new.
2. Import the GitHub repository.
3. Select the Next.js project.
4. Configure production environment variables.
5. Add GEMINI_API_KEY and GEMMA_MODEL.
6. Start deployment.
7. Verify the build completes successfully.
8. Open the public deployment URL.
9. Upload an actual screenshot and test the complete workflow.

Step 7 — Production Verification

Verify:

- No missing environment variables.
- Screenshot uploads work.
- Gemma API calls succeed.
- Result cards render.
- Implementation prompt generation succeeds.
- Downloads contain valid content.
- No API keys are exposed in frontend assets.
- Mobile and desktop layouts work.

A successful deployment build alone is not proof that the application is fully functional.

---

12. Hackathon Demonstration Plan

Recommended Demo Duration: 90 Seconds to 3 Minutes

Step 1 — Introduce the Problem

Developers often receive interface screenshots without knowing the appropriate components, libraries, data models, or implementation steps.

Screenshot2Spec converts these visual references into implementation-ready development blueprints.

Step 2 — Upload a Screenshot

Upload an analytics dashboard or another prepared interface screenshot.

Choose Next.js and Tailwind CSS as development preferences.

Step 3 — Demonstrate Gemma 4

Show how the model identifies visible interface sections, including navigation, cards, charts, and tables.

Explain that Gemma performs the visual understanding, while application logic validates and structures its output.

Step 4 — Show Library Recommendations

Demonstrate how Screenshot2Spec recommends:

- Recharts for data visualizations.
- shadcn/ui for applicable interface components.
- Lucide React for icons.
- Tailwind CSS for layouts.
- Native HTML for simple tables.

Explain why each library was chosen.

Step 5 — Show the Implementation Prompt

Open the generated AI coding prompt.

Highlight:

- Component hierarchy.
- Required dependencies.
- Installation commands.
- Responsive styling instructions.
- TypeScript requirements.
- Testing instructions.

Step 6 — Export the Blueprint

Copy or download the generated implementation prompt.

Explain that developers can provide it, together with the original screenshot, to an AI coding agent.

Step 7 — Conclude

Screenshot2Spec helps developers move from a visual reference to a concrete implementation plan without manually researching every component and library.

---

13. Final Antigravity Audit Prompt

Act as a senior software engineer, QA specialist, and hackathon reviewer.

Perform a complete audit of the Screenshot2Spec repository.

Verify the workflow:

Screenshot → Gemma 4 → Validated JSON → Component Tree → Library Recommendations → Implementation Prompt → Exports.

Review:

1. Shared API contracts.
2. Upload and input validation.
3. Google AI Studio API integration.

4. Zod response validation.
5. Component hierarchy consistency.
6. Approved library catalog.
7. Library installation commands.
8. Generated prompt completeness.
9. TypeScript output correctness.
10. Copy and download functionality.
11. API key security.
12. Mobile responsiveness.
13. Production build.
14. Vercel deployment readiness.
15. README and license.

Run available tests and commands.

Do not claim that any check passed unless it was executed.

Provide:

- Critical bugs.
- Important improvements.
- Optional enhancements.
- Exact files requiring changes.
- Commands executed.
- Test outcomes.
- Final deployment readiness assessment.

Fix only critical issues necessary for the MVP.

Do not add unrelated features.

---

14. Official Resources

Google AI Studio: https://aistudio.google.com/

Gemma on Gemini API: https://ai.google.dev/gemma/docs/core/gemma_on_gemini_api

Google GenAI SDK: https://googleapis.github.io/js-genai/

Next.js: https://nextjs.org/docs

shadcn/ui: https://ui.shadcn.com/docs

Recharts: https://recharts.org/

TanStack Table: https://tanstack.com/table

Vercel: https://vercel.com/docs

---

15. Final Project Success Criteria

The project is successful when a developer can:

1. Open the deployed application.
2. Upload an unfamiliar UI screenshot.
3. Receive a structured Gemma 4 analysis.
4. Explore the proposed component architecture.
5. Understand which frontend libraries are recommended and why.
6. View the corresponding dependency installation commands.
7. Generate a complete implementation prompt.
8. Copy or download that prompt.
9. Use the blueprint as a starting point in an AI coding environment.

Final Project Positioning:

Screenshot2Spec — An AI-powered frontend blueprint generator that transforms UI screenshots into component architectures, verified library recommendations, and implementation-ready coding prompts.

Build the complete core workflow first. A reliable, deployable application is more valuable than an unfinished collection of advanced features.